

109. Animer : deux dessins pour marcher

Un personnage qui glisse sans bouger les jambes a l'air d'un meuble. Deux dessins, alternés, et il marche.

1. Ce qu'il faut comprendre

Une animation, au cinéma comme sur Game Boy, n'est qu'une suite d'images fixes montrées vite. Pour marcher, **deux dessins** suffisent : jambes écartées, jambes serrées.

Ce qui est nouveau ici : le lutin change de dessin. `sprite(0, x, y, PAS1)` ou `sprite(0, x, y, PAS2)` : le même lutin n° 0, au même endroit, mais une autre tuile. La console ne redessine rien : elle lit simplement une autre tuile.

`dessin` vaut 0 ou 1 : lequel des deux montrer. `compte` compte les images ; toutes les 8 images, `dessin = 1 - dessin` le fait **basculer** : $1 - 0 = 1$, puis $1 - 1 = 0$, puis $1 - 0 = 1$... Un interrupteur en une ligne.

Quand aucune flèche n'est enfoncée, on remet `dessin` à 0 : le héros s'arrête **debout**, sur son premier dessin, et pas figé au milieu d'un pas.

Pourquoi 8 images ? À chaque image, les jambes battraient 30 fois par seconde : un tremblement. Toutes les 30 images, le héros aurait l'air de glisser entre deux pas. Entre 6 et 12 images, l'œil voit une marche.

À toi : ajoute un troisième dessin (`PAS3`) et fais tourner `dessin` de 0 à 2 avec `dessin = (dessin + 1) % 3`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **700 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          67 o ← la console, à l'allumage — la mise en route de la console
main()             165 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← démarrage — attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte         10 o ← texte() — écrit un texte, lettre par lettre
RangerLutins        11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles        85 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage — vide les deux cartes de l'écran
LireManette         37 o ← bouton() — lit les huit boutons
```

```

Tuiles          208 o ← CopierTuiles  - 11 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert - la petite routine qui copie les lutins
"GAUCHE DROITE" 14 o ← texte() - un texte gravé
Lettres gravées : ACDEGHIORTU
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes
Fonctions du langage appelées :
    sprite() × 2    46 o sur place, dans main()
    texte() × 1     11 o sur place, dans main() + EcrireTexte (10 o)
    bouton() × 2    10 o sur place, dans main() + LireManette (37 o)
    image() × 1      6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un héros qui MARCHE : quand il avance, ses jambes s'animent
// (deux dessins qui alternent) ; à l'arrêt, il est debout.
//
// Ce qui est nouveau : changer le dessin d'un lutin. Le même lutin n° 0,
// au même endroit, reçoit tantôt la tuile PAS1, tantôt la tuile PAS2.

// Deux dessins : jambes écartées, jambes serrées. Le haut est le même.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près

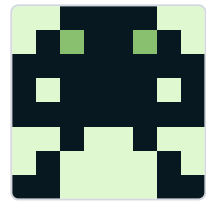
```

2. UN DESSIN — PAS1

```

Tuile PAS1 = {
    "..####.",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};

```



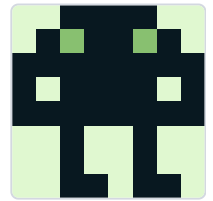
à l'écran

3. UN DESSIN — PAS2

```

Tuile PAS2 = {
    "..####.",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "..###.",
};

```



à l'écran

4. UNE VARIABLE — X

```

uint8_t x = 76;    // La place du héros, en pixels.

```

5. UNE VARIABLE — DESSIN

```

uint8_t dessin = 0;    // 0 : PAS1, 1 : PAS2

```

6. UNE VARIABLE — COMPTE

```

uint8_t compte = 0;    // les images depuis le dernier changement de dessin

```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(1, 1, "GAUCHE  DROITE");

    while (true) {                          // La boucle du jeu :
        image();

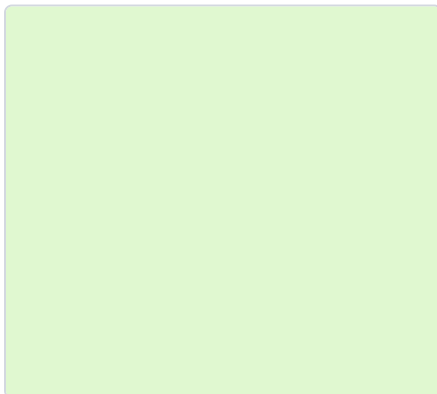
        uint8_t bouge = 0;                  // 1 si le héros avance cette image
        if (bouton(DROITE) && x < 152) { x++; bouge = 1; } // Deux instructions entre
        if (bouton(GAUCHE) && x > 0)  { x--; bouge = 1; } // accolades : les deux se font.

        if (bouge) {                        // Il marche :
            compte++;
            if (compte >= 8) {               // toutes les 8 images ...
                compte = 0;
                dessin = 1 - dessin;        // ... on bascule : 0 -> 1 -> 0 -> 1
                                              // (1 - 0 = 1, puis 1 - 1 = 0 : un interrupteur)
            }
        } else {                            // Il ne marche pas :
            dessin = 0;                     // à l'arrêt : debout, sur le premier dessin
            compte = 0;
        }

        if (dessin == 0) {                  // On pose le lutin avec le bon dessin.
            sprite(0, x, 72, PAS1);
        } else {
            sprite(0, x, 72, PAS2);
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



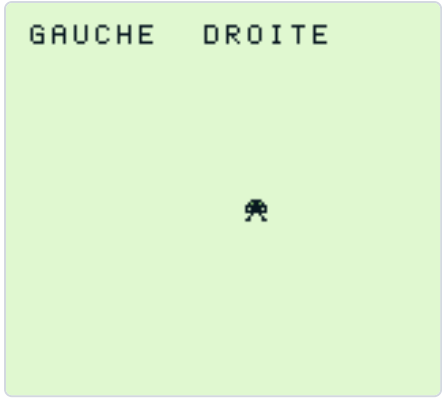
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un héros immobile ; avec DROITE ou GAUCHE, ses jambes battent pendant qu'il avance.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ en marchant, le lutin montre deux dessins (2)
- ✓ à l'arrêt, il revient à son premier dessin

Ensuite : **110. Regarder à gauche ou à droite : le miroir** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++